

Database Systems I / Information Systems I

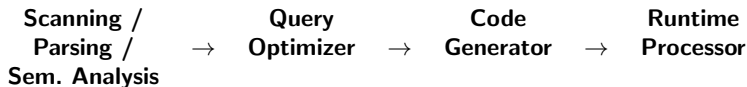
Query Processing and Optimization — Complementary Material

Christian Rauch

(changed: 2026-06-29)

Query Processing Pipeline

A SQL query passes through four stages before any data is touched:



Scanning / Parsing Tokenise the SQL text; build an abstract syntax tree (AST). Syntax errors are caught here.

Semantic Analysis Resolve table and column names against the schema; check types. Semantic errors (unknown table, type mismatch) are caught here.

Query Optimizer Rewrite the query into an efficient *query plan* – the central topic of this section.

Code Generator Translate the plan into executable operators.

Runtime Processor Execute operators, fetch pages, return tuples.

Clustered vs. Unclustered Relations

How records are laid out on disk fundamentally affects scan cost.

Clustered relation

Records are stored packed into consecutive blocks. If b records fit per block and the relation has $T(R)$ records:

$$B(R) = \lceil T(R)/b \rceil \text{ blocks} \Rightarrow B(R) \text{ I/Os to scan}$$

Unclustered (scattered) relation

Records are spread across blocks shared with other tables. In the worst case every record is on a different block:

$$\text{scan cost} = T(R) \text{ I/Os}$$

Since $T(R) \gg B(R)$, an unclustered scan is drastically more expensive.

Clustered vs. Unclustered Relations (cont.)

- ▶ A table has **at most one** clustering order on disk – records can only be in one physical order.
- ▶ Secondary (non-clustering) index scans incur the scattered cost $T(R)$ for low-selectivity queries.
- ▶ The asymmetry drives the outer-relation choice in nested-loop joins: the unclustered relation should be outer so its expensive scan is paid only once.

Linear Search

The simplest scan: read every block of the file and check each record against the predicate.

Cost

b block I/Os, where b is the number of blocks in the file. Average cost for a *unique match*: $b/2$ I/Os.

- ▶ Works for *any* predicate on *any* attribute – no index required.
- ▶ Only viable strategy when no usable index exists or when most records satisfy the predicate.
- ▶ Always correct; used as the fallback baseline.

Binary Search

Applicable when the file is **physically sorted** on the search attribute and the predicate is an equality or range.

Cost

$\lceil \log_2 b \rceil$ I/Os to locate the first matching block, then sequential reads for range results.

- ▶ Much faster than linear search for large files ($\log_2 10^6 \approx 20$ vs. 10^6 I/Os).
- ▶ Requires the file to stay sorted – inserts and updates are expensive (shifted records).
- ▶ Rarely used directly in modern DBMS; superseded by B^+ tree indexes over sorted files.

Primary Hash Table Search

Use a **hash index** on the search attribute.

Equality predicate only (`WHERE id = ?`)

Compute $h(k)$ to get the bucket address. Read the bucket block (and any overflow chain). **Cost: 1 I/O** (no overflow), $1 + c$ I/Os (overflow chain of length c).

- ▶ *Primary* means the file is organised by the hash key – the index points to the actual records, not just their locations.
- ▶ Range predicates are **not supported**: hash destroys order.
- ▶ Optimal when equality lookups dominate and the table is stable.

Clustering Index Search

The **data pages** (the blocks that store the actual records) are physically ordered on disk by the index key. The index (typically a B^+ tree) does not just point to records – it reflects their actual layout on disk.

1. Traverse the B^+ tree to the first leaf matching the predicate: $\lceil \log_t b \rceil$ I/Os (t = tree order, b = file blocks).
2. Read blocks *sequentially* from that point: all matching records are guaranteed to be in **adjacent blocks** because the file order matches the index order.

A disk (or OS prefetcher) can stream adjacent blocks very efficiently. With a secondary (non-clustering) index the same records could be scattered across many blocks, requiring one I/O each.

- ▶ A table can have **at most one** clustering index – records can only be in one physical order.
- ▶ Ideal for range queries; equality lookup costs $\lceil \log_t b \rceil + 1$ I/Os.

Secondary Index Search

A **secondary** (non-clustering) index is built on an attribute that does *not* determine physical record order.

Cost – the “scattered read” problem

Each matching record may reside in a *different* block. Worst case: **one I/O per matching record** – potentially worse than a linear scan for large result sets.

- ▶ A table can have **many** secondary indexes.
- ▶ Effective for *highly selective* queries (few matching records).
- ▶ For low-selectivity queries the optimizer may choose a linear scan instead – secondary index lookup would cost more.

Composite Index

A **composite** (multi-attribute) index is built on an ordered list of attributes, e.g. (A, B) . Records are sorted first by A , then by B within each A value.

Leftmost prefix rule: The index guides a scan only for predicates forming a *prefix* of the key: equality on all leading columns, then at most one range on the next column. Columns after a range predicate must be filtered in memory.

Predicate on (A, B)	Index usable?	Reason
$A = 5 \text{ AND } B = 7$	fully	exact prefix match
$A = 5 \text{ AND } B \text{ BETWEEN } 1 \text{ AND } 9$	fully	equality on A , range on B
$A \text{ BETWEEN } 1 \text{ AND } 5$	fully	range on leading column
$A \text{ BETWEEN } 1 \text{ AND } 5 \text{ AND } B = 7$	partial	range stops index use for B
$B = 7$	not at all	B is not globally sorted

- ▶ Put the most selective equality attribute *first* in the key.
- ▶ A composite index also covers queries on the leading column(s) alone.
- ▶ One composite index on (A, B) is often better than two separate indexes.

Conjunctive Queries (AND)

WHERE c_1 AND c_2 AND $\dots$

Strategy – use the most selective condition first:

1. If an index exists for one of the conditions, use it to retrieve a candidate set.
2. Filter the candidate set in memory against the remaining conditions.

Composite index

If an index covers *all* conditions (multi-attribute key), the entire conjunction can be evaluated in the index – no post-filtering needed.

- ▶ Apply the *most selective* predicate first to minimise the intermediate result size.
- ▶ Selectivity = $|\sigma_c(r)|/|r|$ – smaller is better.

Disjunctive Queries (OR)

WHERE c_1 OR c_2 OR $\dots$

Challenge

Every condition can independently match records – the result is the *union* of all matching sets. A single selective index cannot prune the scan.

Options:

- ▶ If indexes exist for *all* conditions: retrieve each set separately and take their union (de-duplicate).
- ▶ If any condition has no index: a **linear scan** is unavoidable – every record must be checked.

Disjunctions are therefore harder to optimise than conjunctions; query rewriters sometimes convert them to UNION queries to allow per-branch index use.

Join – Overview

The join $R \bowtie_{\theta} S$ is typically the most expensive operation in a query. Cost depends on:

- ▶ Sizes $|R|$, $|S|$ (in blocks: b_R , b_S).
- ▶ Available **buffer** (memory pages M).
- ▶ Availability of **indexes** on the join attributes.
- ▶ Nature of the join predicate ($=$, $<$, arbitrary θ).

Algorithm	Best for
Nested-loop join	Small tables, arbitrary θ
Index-based join	Index on inner relation
Sort-merge join	Large tables, equality, sorted input
External hash join	Large tables, equality, unsorted

Nested-Loop Join

For each tuple $r \in R$ (outer), scan all of S (inner) for matches.
Read R in chunks of $M-1$ blocks; for each chunk scan S once.

$$\text{Cost} = \text{scan}(R) + \left\lceil \frac{B(R)}{M-1} \right\rceil \cdot B(S)$$

where $\text{scan}(R)$ is the cost of reading R once (clustered or not) and $B(R)$ is its block count when clustered.

The inner S is re-read $\lceil B(R)/(M-1) \rceil$ times; the outer is read only once. Works for **any** join predicate θ ; no index required.

Nested-Loop Join – Outer Relation Choice

Rule: make the **unclustered** relation the **outer**

If one relation is **unclustered**, place it as the **outer**. The outer is scanned *once*, paying its full unclustered cost $T(R)$ exactly once. As the inner, that scan is repeated for every chunk of the outer:

$$\underbrace{B(S) + \left\lceil \frac{B(S)}{M-1} \right\rceil \cdot T(R)}_{\text{unclustered } R \text{ as inner}} \gg \underbrace{T(R) + \left\lceil \frac{B'(R)}{M-1} \right\rceil \cdot B(S)}_{\text{unclustered } R \text{ as outer}}$$

where $B'(R) = T(R)/b$ is the packed block count of R .

If both relations are clustered, prefer the **smaller** as outer to minimise the chunk count and thus the number of inner re-scans.

Index-Based Join

Requires an index on the join attribute of the **inner** relation S .
For each tuple $r \in R$: use the index on S to retrieve matching tuples directly.

Cost

$B(R)$ I/Os to scan R + index lookup cost per r -tuple.

- ▶ **Clustering index on S** : $\approx B(R) + \lceil \log_t B(S) \rceil \cdot T(R)$ I/Os.
- ▶ **Secondary index on S** : up to one I/O per matching tuple in S – can be costly if many matches.
- ▶ Best when $T(R)$ is small and the index is highly selective.
- ▶ Only supports **equality** (and some range) join predicates.

Sort-Merge Join

1. **Sort** R and S independently on the join attribute (using external merge sort – see next slide).
2. **Merge-scan:** stream through both sorted relations once, advancing the smaller pointer and emitting matches.

Pass counting for a relation of B blocks with buffer M

Pass 1 (run generation): load M blocks, sort in memory, write.
Produces $\lceil B/M \rceil$ runs at cost $2B$ I/Os.

Further passes: merge $M-1$ runs at a time, cost $2B$ each.
Number of merge passes: $\lceil \log_{M-1} \lceil B/M \rceil \rceil$.

Total: $2B \cdot (1 + \lceil \log_{M-1} \lceil B/M \rceil \rceil)$ I/Os.

Join cost: sort R + sort S + merge-scan ($B(R) + B(S)$ I/Os).
Output is sorted – beneficial if ORDER BY is needed.

Sort-Merge Join – Worked Example

$$B(R) = 15,000, B(S) = 10,000, M = 101.$$

Sorting R

Pass 1: $\lceil 15,000/101 \rceil = 149$ runs.

Pass 2: $\lceil 149/100 \rceil = 2$ runs.

Pass 3: $\lceil 2/100 \rceil = 1$ run. $\Rightarrow$ 3 passes total.

Cost: $15,000 \times 3 \times 2 = 90,000$ I/Os.

Sorting S

Pass 1: $\lceil 10,000/101 \rceil = 100$ runs.

Pass 2: $\lceil 100/100 \rceil = 1$ run. $\Rightarrow$ 2 passes total.

Cost: $10,000 \times 2 \times 2 = 40,000$ I/Os.

Merge-scan

One sequential pass over both sorted relations:

$15,000 + 10,000 = 25,000$ I/Os.

$$\text{Total} = 90,000 + 40,000 + 25,000 = \mathbf{155,000} \text{ I/Os}$$

External (Disk-Based) Sorting

Required by sort-merge join when relations do not fit in memory.

External merge sort – two phases:

1. **Run generation:** read M blocks at a time, sort in memory, write a *sorted run* to disk. Produces $\lceil B/M \rceil$ runs.
2. **Merge:** merge up to $M-1$ runs at a time using one output buffer; repeat until one run remains.

Total I/O cost (for a relation of B blocks)

$2B \cdot (1 + \lceil \log_{M-1} \lceil B/M \rceil \rceil)$ (each pass reads and writes every block once).

With $M = 3$ pages this degenerates to a two-way merge. Larger buffers reduce the number of merge passes dramatically.

Hash Join

Exploits the fact that matching tuples hash to the *same bucket*.

Two phases:

1. **Partition:** use 1 buffer slot to read input and $M-1$ slots as output buffers, one per partition. Hash both R and S into $M-1$ partition files using h_1 . Matching tuples always land in the same partition.
2. **Probe:** for each partition i , load S_i into $M-1$ buffer slots, build an in-memory hash table (using $h_2 \neq h_1$), then stream R_i through the remaining 1 slot and probe for matches.

Cost

Partition: $2(B(R) + B(S))$ I/Os (read + write both relations).

Probe: $B(R) + B(S)$ I/Os (read all partitions once).

Total: $3(B(R) + B(S))$ I/Os.

Hash Join – Memory Requirement

How large must the buffer be?

In the probe phase, S_i must fit entirely in $M-1$ buffer slots (1 slot is reserved for streaming R_i block by block).

With $M-1$ partitions and a uniform hash, each S_i has $B(S)/(M-1)$ blocks. Fitting in $M-1$ slots requires:

$$\frac{B(S)}{M-1} \leq M-1 \implies B(S) \leq (M-1)^2$$

Example: $B(S) = 10,000$, $M = 101$

$(M-1)^2 = 100^2 = 10,000 = B(S)$ – fits exactly.

Each of the 100 partitions of S has 100 blocks, filling the 100 available probe-phase slots precisely.

Only **equality** join predicates are supported.

Query Optimization – Overview

The optimizer transforms a *logical query plan* (relational algebra tree) into an efficient *physical query plan* (choice of algorithms and access paths).

Heuristic Apply rule-based rewrites to the logical plan before cost estimation.

Cost-based Enumerate candidate physical plans; estimate I/O and CPU cost for each; pick the cheapest.

Most real optimizers combine both: heuristics reduce the search space, cost estimation selects among the remaining candidates.

Heuristic (Logical) Optimization

Applied to the **relational algebra tree** before choosing physical operators. Key rules:

Push selections down (“apply early”)

Move σ as far down the tree as possible – filter rows *before* joins to shrink intermediate results.

$$\sigma_c(R \bowtie S) \equiv \sigma_c(R) \bowtie S \quad (\text{if } c \text{ only references } R)$$

Push projections down

Apply π early to drop unused columns before joining – reduces tuple width and I/O.

Other common rewrites:

- ▶ Replace Cartesian product $\times + \sigma$ with $\bowtie$.
- ▶ Reorder joins so the smallest intermediate result is produced first.
- ▶ Combine cascaded selections: $\sigma_{c_1}(\sigma_{c_2}(R)) = \sigma_{c_1 \wedge c_2}(R)$.

Cost-Based (Physical) Optimization

After heuristic rewrites, the optimizer assigns **physical operators** and estimates cost.

Statistics used

- ▶ $|R|$, b_R – tuple count and block count.
- ▶ $V(A, R)$ – number of distinct values of attribute A .
- ▶ Histograms for selectivity estimation.

Statistics are stored in the system catalogue and updated by
ANALYZE / UPDATE STATISTICS.

Plan space: join order alone has $n!$ permutations for n tables – the optimizer uses dynamic programming (System R approach) to find the optimal left-deep join tree without enumerating all plans.

Final plan selects, per operator, the best of: linear scan, clustering-index scan, secondary-index scan, nested-loop / index / sort-merge / hash join.